

Final Project - Data Science in Cyber

1. Summary of the Source

The Problem Being Addressed

Phishing remains a premier social engineering threat vector in cybersecurity, wherein adversaries construct fraudulent web domains mimicking trusted corporate entities to harvest sensitive user credentials, financial documentation, or deliver malicious payloads.

Why the Problem is Important

Traditional defensive perimeters rely heavily on static blacklisting (e.g., Google Safe Browsing, PhishTank). While operationally fast, blacklists exhibit low structural resilience against Zero-Day phishing attacks newly generated, short-lived URLs that haven't been indexed yet. Machine Learning offers a proactive mechanism to classify links via underlying behavioral structural footprints.

The Proposed Solution

The source implements an automated URL feature-extraction framework paired with a supervised multi-classifier comparison system. The goal is to ingest raw URL strings, split out structural categorical weights, and classify whether a target domain is legitimate (0) or phishing (1).

The Dataset Used

The original work compiles an aggregated dataset comprising 10,000 distinct URLs:

- **5,000 Phishing URLs:** Sampled from PhishTank.
- **5,000 Legitimate URLs:** Sampled from the University of New Brunswick (CIC-URL-2016 dataset). The compiled features are packed statically into 5.urldata.csv.

The Model/Methodology Employed

The author performs manual rule-based parsing on raw text to extract 17 distinct attributes spanning:

- Address Bar features (e.g., URL Length, IP tracking symbols, @ metrics)
- Domain indicators (e.g., DNS verification, Domain Age)
- HTML/JavaScript triggers (e.g., IFrame usage, Right-click disabling status)

The data is subsequently passed to six supervised models: Decision Trees, Random Forests, Multilayer Perceptrons (MLP), XGBoost, Support Vector Machines (SVM), and an Autoencoder network.

2. Critical Evaluation

The Main Claims Made by the Author

The author asserts that extracting lightweight structural properties directly from the URL allows for robust, accurate classification without the overhead of heavy server-side deep inspections, claiming XGBoost provides the premier deployment paradigm with an optimal internal verification performance (~86.4%).

Evidence & Evaluation Methodology Critiques

While the empirical modeling outputs appear sound on paper, a deeper methodological audit reveals major, foundational cracks that break the real-world cybersecurity utility of this work:

- **Artificial Class Prevalence (Sampling Bias):** The source relies on a perfectly balanced 50\% to 50\% split (5,000 legitimate vs. 5,000 phishing links). In actual production environments, malicious URLs are extreme, high-skew anomalies—often comprising less than 0.1\% of enterprise network logs. By ignoring real-world class imbalance, the author's precision metrics are artificially inflated.
- **Lack of Shuffling or Cross-Validation Rigor:** The original dataset stores the targets sequentially (0 entries stacked ahead of 1 entries). If an evaluator runs a primitive split without active shuffling (shuffle=True), the training matrix isolates benign structures exclusively, leaving the model incapable of identifying malicious patterns during subsequent testing phases.

Possible Weaknesses or Limitations

The Domain-Name Omission Paradox: In the author's primary pipeline development, the literal Domain string attribute is entirely dropped. This represents a monumental disconnect from active cybersecurity threat models. Dropping domain data deprives the system of any ability to spot Typosquatting or Homograph attacks (e.g., identifying micros0ft.com vs microsoft.com).

Brittleness of Static Thresholding: The author converts continuous attributes into binary attributes using hardcoded rules (e.g., labeling any URL ≥ 54 characters as malicious). Modern corporate links heavily leverage complex tracking parameters (UTM strings) that naturally dwarf 54 characters, ensuring this model would collapse under an explosive flood of False Positives in an active enterprise framework.

Whether the Conclusions are Justified

The author's conclusions are not fully justified. While they correctly conclude that machine learning can find patterns in URL text, their conclusion that this model is ready for real-world deployment (like a browser extension) is deeply flawed. Because they trained the model on a highly artificial 50/50 balanced dataset using hardcoded rules, the model's high accuracy is an illusion. In a live environment, it would trigger an unmanageable flood of False Positives, blocking users from legitimate websites and forcing security teams to disable the tool. Therefore, their success metrics do not reflect practical performance.

3. Feature Engineering Analysis

Whether Feature Engineering Was Performed & Which Features Were Used

Yes, feature engineering was performed by the author, but it was done entirely as a manual, rule-based text-parsing operation. The pipeline maps raw URL strings into 18 specific features stored in 5.urldata.csv:

- **Target Label:** `Label` (0 = Legitimate, 1 = Phishing)
- **Text/Identifier Input:** `Domain` (Dropped during training)
- **Address Bar Features:** `Have_IP`, `Have_At`, `URL_Length`, `URL_Depth`, `Redirection`, `https_Domain`, `TinyURL`, `Prefix/Suffix`
- **Domain-Based Features:** `DNS_Record`, `Web_Traffic`, `Domain_Age`, `Domain_End`
- **HTML/JS Features:** `iFrame`, `Mouse_Over`, `Right_Click`, `Web_Forwards`

Analysis of Transformations, Normalization, and Encoding

A major weakness of the original work is the complete absence of standard mathematical data science transformations.

Advanced Transformations (Logarithmic, Box-Cox, Standardization, Min-Max Scaling): These were not applied by the author. Features like `URL_Length`, `URL_Depth`, `Domain_Age`, and `Web_Traffic` are naturally continuous variables with highly skewed distributions. Instead of applying a Logarithmic or Box-Cox transformation to normalize these spreads, or using Min-Max/Standardization scaling to bring them into a shared variance, the author manually compressed them into static binary integers (0 or 1) or small arbitrary ranges (e.g., mapping length to 0 or 1 based on a hard boundary of 54 characters).

Impact on Performance: This lack of scaling and normalization heavily degrades the performance of linear or distance-based models like Logistic Regression and SVM. It strips away structural variance, presenting the models with jagged, artificial data boundaries. Only tree-based ensembles (like Random Forest and XGBoost) can navigate these rough steps

because they split data on discrete thresholds rather than calculating gradients or geometric distances.

Categorical Encoding (One-Hot vs. Target Encoding): The author did **not** apply proper One-Hot Encoding or Target Encoding. They treated ordinal variables (like custom threat levels assigned to feature columns) as continuous integers. This introduces a mathematical flaw where a model might assume a value of 1 is double the weight of 0, distorting the internal mathematical intuition of linear algorithms.

System Redundancy: Detection & Remediation

System redundancy occurs when features provide overlapping information, increasing computational cost and risk of overfitting.

How to Spot It: By generating a non-parametric Spearman Rank Correlation Heatmap, we can observe high co-linearity scores ($\rho \geq 0.80$) between distinct inputs. For instance, features capturing technical anomalies like iFrame visibility modifications, custom status-bar scripts, and Mouse_Over actions frequently showcase extreme mathematical overlap because phishing kits typically deploy these JS scripts together.

Tackling the Issue: Redundant properties can be pruned via automated Recursive Feature Elimination (RFE) or penalization models like Lasso (L1 regularization) to strip out highly co-linear, low-variance dependencies.

Mathematical vs. Cybersecurity Intuition Assessment

The author's engineering choices lack solid mathematical foundation. Features like URL_Length and URL_Depth represent skewed, continuous measurements, yet they are compressed into arbitrary binary buckets. From a data science perspective, this destroys structural variance and introduces jagged decision boundaries that confuse linear classifiers like Logistic Regression.

Cybersecurity threat groups have actively mutated their techniques to bypass the rigid boundaries encoded in this dataset:

- **Mathematical Assumption:** Phishing pages utilize deep directories (URL_Depth) and elongated strings (URL_Length).
- **Cybersecurity Reality:** Modern adversaries extensively deploy shortened redirectors (e.g., bit.ly) or sub-domain takeovers on legitimate, high-reputation base sites. In these scenarios, a phishing link is short and shallow, allowing it to bypass the author's feature criteria entirely and generate a catastrophic False Negative.

Additional Features to Improve Performance

To modernize this detection approach, the feature engineering pipeline should be updated to include:

- **Lexical Levenshtein Distances:** Calculate the string edit distance between the target domain name and top-1000 highly targeted corporate financial brands to programmatically identify typosquatting attempts.
- **Entropy of Domain String:** Compute Shannon Entropy (H) on the domain text string:

$$H = \sum_{i=1}^n p(x_i) * \log_2 p(x_i)$$

High entropy scores flag randomized, algorithmically generated domain names (DGAs) commonly utilized by threat actors spinning up short-lived phishing nodes.

4. Reproducibility Analysis

Whether the Code Can Be Executed Successfully

The reproducibility of this repository depends entirely on which notebook is executed:

- **Phishing Website Detection_Models & Training.ipynb** : This notebook executes successfully without adjustments, provided the static file 5.urldata.csv is locally available in the working directory. It handles standard matrix operations via pandas and passes clean arrays to scikit-learn and xGboost.

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, roc_auc_score

# Drop string column and isolate target
X = df.drop(columns=['Domain', 'Label'])
y = df['Label']

# 80-20 Train-Test Split with a fixed random seed
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=12, shuffle=True)

# Initialize and train
rf = RandomForestClassifier(random_state=42)
lr = LogisticRegression(max_iter=1000, random_state=42)

rf.fit(X_train, y_train)
lr.fit(X_train, y_train)

# Print verification out
print(f"Random Forest Accuracy: {accuracy_score(y_test, rf.predict(X_test)):.4f}")
print(f"Logistic Regression Accuracy: {accuracy_score(y_test, lr.predict(X_test)):.4f}")

... Random Forest Accuracy: 0.8705
    Logistic Regression Accuracy: 0.8065
```

- **URL Feature Extraction.ipynb (Fails)**: This notebook cannot be executed successfully from scratch. It is designed to take the raw text strings from 1.Benign_list_big_final.csv and 2.online-valid.csv and dynamically extract features over the live internet. When run today, the script hangs indefinitely or crashes.


```
import urllib.request

# Target an actual URL from the original dataset used in 2016
test_url = "http://graphicriver.net"

print(f"Attempting live feature extraction connection to: {test_url}...")

try:
    # This mimics the author's core feature extraction step:
    # sending a web request to scan the page live over the internet.
    response = urllib.request.urlopen(test_url, timeout=5)
    print("Success! Response Code:", response.getcode())

except Exception as e:
    # If the domain is dead, changed, or blocks automated bots, it crashes here.
    # We catch the crash and print this message out as scientific evidence for your report.
    print("\n[REPRODUCIBILITY ERROR CAUGHT]:")
    print(f"The connection failed with error: {e}")
```

... Attempting live feature extraction connection to: <http://graphicriver.net>...

[REPRODUCIBILITY ERROR CAUGHT]:
The connection failed with error: HTTP Error 403: Forbidden

Availability of Required Files and Dependencies

Files: The author successfully provided all intermediate data steps. The raw URL lists, the isolated individual features (legitimate.csv, phishing.csv), and the final combined matrix (5.urldata.csv) are all present in the DataFiles folder.

Dependencies: The repository contains a critical engineering omission: **there is no requirements.txt or environment configuration file.** The code relies heavily on external libraries like python-whois, urllib, BeautifulSoup4, scikit-learn, and xGboost. Because version numbers are not pinned, modern versions of these libraries throw deprecation warnings or syntax errors when processing her older code structural patterns.

Hidden Preprocessing Steps

While the feature extraction logic is documented inside the extraction notebook, several hidden execution assumptions exist:

- **Lack of Exception Handling for Dead Domains:** The code assumes that every URL in the historical dataset is still live and active on the internet. In reality, when `urllib.request.urlopen()` or `whois.whois()` queries an expired domain, it triggers long socket timeouts or returns empty/error responses. The author's code has no try-except safety nets to handle these network dropouts gracefully, hiding the fact that reproduction requires an internet connection untouched by time.
- **Silent Domain Dropping:** The dropping of the Domain column happens silently at the beginning of the modeling notebook without exploring whether character embeddings or text vectorization (e.g., TF-IDF) could preserve its context.

The Overall Reproducibility of the Work

The overall reproducibility of this work is medium-to-low.

If a data scientist attempts to reproduce the pipeline from its true starting point (the raw URLs) reproduction is completely broken because the live internet has mutated since the project was uploaded. Dead domains and changed DNS records make it impossible to recreate the exact same feature matrix dynamically.

However, the downstream machine learning component is highly reproducible. Because the author provided the static intermediate file `5.urldata.csv`, we can completely bypass the broken extraction script, load the static data directly, and successfully verify, audit, and retrain the models.

5. Experimental Results

The Experiments Performed and Models Trained

The experimental phase was structured as a head-to-head architectural validation study on the 10,000-sample 5.urldata.csv matrix. The core experiment evaluated how two fundamentally different mathematical frameworks handle discrete, pre-processed cybersecurity indicator features.

The Primary Model: A non-linear ensemble architecture (**Random Forest Classifier**) was trained to evaluate its ability to build decision tree pathways over multi-valued integer steps and binary indicators.

The Baseline Model: A linear architecture (**Logistic Regression**) was trained to measure performance boundaries when forcing a continuous optimization gradient across discrete step data.

Modifications Introduced

To elevate the codebase to rigorous academic standards and prevent the engineering flaws found in the original open-source tutorial, three specific pipeline modifications were introduced:

- **Global Seed Hardening:** Pinned a global `RANDOM_SEED = 42` across both the split and the model initializers to guarantee 100% deterministic, reproducible results.
- **Forced Random Shuffling:** Implemented a mandatory data shuffle during the split phase to break the sequential block ordering of the raw data (where the first 5,000 rows were completely benign and the last 5,000 were completely malicious).
- **Target Stratification:** Enforced target vector stratification (`stratify=y`) to guarantee that the 50/50 class distribution was perfectly

maintained across both the 8,000-row training set and the 2,000-row verification set, preventing sampling split bias.

Evaluation Metrics and Obtained Results

The models were evaluated using an array of classification metrics to capture operational cybersecurity trade-offs. The empirical results compiled from the 2,000-sample testing partition are detailed in the performance table below:

Evaluation Metric	Random Forest (Ensemble)	Logistic Regression (Baseline)
Accuracy	0.8670 (86.70%)	0.7975 (79.75%)
Precision	0.9248 (92.48%)	0.9268 (92.68%)
Recall (Sensitivity)	0.7990 (79.90%)	0.6460 (64.60%)
F1-Score	0.8573	0.7613
F-Beta ($\beta = 0.5$)	0.8965	0.8527
Matthews Correlation Coefficient (MCC)	0.7409	0.6244

ROC-AUC	0.9258	0.8740
Confusion Matrix (TN, FP, FN, TP)	(935, 65, 201, 799)	(949, 51, 354, 646)

The ensemble model demonstrated clear superiority across almost all dimensions. The Random Forest achieved a high ROC-AUC of 92.58% and successfully caught 799 true threats. Logistic Regression struggled significantly with the data layout, dropping to a 79.75% overall accuracy and experiencing massive classification failure by missing 354 active attacks.

6.Conclusions

Key Findings

The primary finding of this reproduction study is that non-linear, tree-based models are significantly better suited for signature-derived cybersecurity matrices than linear models. Logistic Regression failed because linear equations expect smooth, continuous variables; forcing a linear gradient across rigid, pre-binned values like \$0\$ and \$1\$ distorts the decision boundary. Conversely, Random Forest naturally thrives on discrete data by using orthogonal splits.

However, looking past high-level metrics reveals that statistical significance does not mean practical security significance. Despite an 86.70% accuracy, the Random Forest model suffered from a dangerous **20.10% False Negative rate** (missing 201 threats), which makes it completely unsafe for a real-world enterprise environment.

Lessons Learned

This project highlights a vital data science lesson: evaluating a security model solely on an artificially balanced (50/50) dataset creates a false sense of security. In production networks, phishing links make up less than 0.1% of live logs. A model with a 20.10% leakage rate deployed in a real-world environment would allow thousands of malicious links to bypass the firewall daily, leading to widespread credential theft and network compromise.

Strengths and Weaknesses of the Proposed Solution

Strengths: The system has low computational overhead and trains incredibly fast because the 17 features are highly compressed integers. It also maintains excellent **Precision (over 92%)**, meaning that when the model *does* block a link, it is highly accurate, preventing false alarms and alert fatigue for security operations teams.

Weaknesses: The pipeline relies on an easily bypassed feature architecture. By completely dropping the raw text string of the **Domain** column without extracting its linguistic properties, the pipeline discards critical contextual indicators. Attackers can easily bypass this entire firewall by using shortened URLs or hosting deep directories on trusted content delivery networks (CDNs).

Suggestions for Future Improvements

To transform this educational pipeline into a resilient, enterprise-grade solution, three major improvements are required:

- **Natural Language Processing (NLP):** Instead of dropping the raw Domain string, character-level TF-IDF vectorization or character embeddings should be implemented to catch typosquatting patterns (e.g., paaypal.com).
- **Behavioral and Contextual Features:** Integrate real-time threat intelligence metadata, such as domain registration age (WHOIS data) and SSL/TLS certificate validity metrics.
- **Dynamic Threshold Tuning:** Shift away from a rigid 0.5 decision boundary and use the ROC-AUC curve to actively tune the model for high recall, intentionally sacrificing some precision to minimize dangerous False Negatives.

Part 7 Executive Summary

This project presents a reproduction study and critical security audit of an open-source machine learning pipeline designed to combat phishing. In modern cybersecurity, phishing URLs serve as the primary initial access vector for threat actors executing credential theft, business email compromise, and ransomware campaigns. The technical challenge addressed here is determining whether supervised machine learning classification algorithms can reliably identify malicious web links based entirely on static structural signatures before a user visits the page. To evaluate this approach, this study replicates a widely utilized GitHub security tutorial using the standardized 5.urldata.csv dataset, which contains a perfectly balanced sample structure of 5,000 legitimate (benign) links and 5,000 phishing (malicious) links.

The data engineering methodology established a highly structured and repeatable data architecture to evaluate the original pipeline. First, data selection was executed by programmatically stripping away non-numeric data components, specifically the raw string Domain column to satisfy the matrix processing requirements of standard estimators. Next, an 80/20 train-test split was applied to the 10,000 observations using a globally pinned random seed to guarantee mathematical reproducibility. Crucially, stratification was enforced to maintain the baseline 50/50 class distribution across both the 8,000-sample training matrix and the 2,000-sample verification partition. To conduct a thorough performance audit, a non-linear ensemble (Random Forest Classifier) was pitted head-to-head against a linear baseline (Logistic Regression) across a comprehensive suite of classification metrics.

The reproduction study yielded definitive performance metrics across the test partition. The non-linear Random Forest model proved to be the superior architecture, achieving an overall **Accuracy of 86.70%**, **Precision of 92.48%**, **Recall of 79.90%**, an **F1-Score of 85.73%**, and a highly robust **ROC-AUC of 92.58%**. Conversely, the linear Logistic Regression model struggled significantly on the discrete dataset, dropping to **79.75% Accuracy**, **92.68% Precision**, **64.60% Recall**, and an **87.40% ROC-AUC**.

The raw confusion matrices, (935 TN, 65 FP, 201 FN, 799 TP) for Random Forest and (949 TN, 51 FP, 354 FN, 646 TP) for Logistic Regression, reveal that while both models maintain strong precision (low false alarms), they suffer from a substantial vulnerability in threat leakage.

The empirical data gathered supports the author's primary mathematical claims: tree-based ensembles are capable of isolating predictive rules from this specific feature matrix. However, the deeper data science insights obtained during our analysis reveal that this performance is highly artificial. A key mathematical insight occurred during the correlation phase, where Pearson, Spearman, and Kendall metrics converged identically to -0.541287 for the length attribute. This serves as empirical proof that the data was heavily pre-compressed into binary indicator states, which strips away the natural variance found in live web traffic. Furthermore, the linear model's failure highlights that linear equations cannot map a smooth mathematical gradient across hardcoded, discrete integer step functions.

Based on the security implications of the error profiles, I explicitly do not recommend using this project architecture for real-world phishing detection problems. While the code provides an excellent educational framework for machine learning, its reliance on manual, static structural rules leaves it highly vulnerable to adversarial evasion. An attacker can easily bypass this entire feature layer by utilizing URL shorteners or hosting deep, nested directories on highly trusted content delivery networks (CDNs).

In conclusion, while the project code is technically valid and successfully reproduced, its statistical significance does not translate to practical, real-world security significance. To protect production enterprise environments, the architecture must be modernized to incorporate Natural Language Processing (NLP) for character-level domain strings, live domain-age reputation lookups, and active page content analysis rather than relying strictly on static URL properties.